

Auditoría Profunda: Login · Llamadas · Mensajes

Revisión detallada de las 3 áreas principales. Se encontraron 9 bugs nuevos críticos — incluyendo problemas que hacen que las llamadas no suenen, no se conecten, tengan eco, o se corten. Código corregido listo para reemplazar.

#	Área	Problema	Sev.
1	Login	reloadUser() no usa await correctamente — puede no sincronizar	CRÍTICO
2	Login	Al entrar con Google, PermissionActivity puede navegar a ChooseNumber innecesariamente	ALTA
3	Llamadas	FCM: offer SDP no se incluye en el data payload — receptor no puede contestar	CRÍTICO
4	Llamadas	ForegroundService arranca ANTES de que haya oferta — ringtone suena pero no hay llamada	ALTA
5	Llamadas	AudioManager no solicita foco de audio — otro audio del sistema interfiere	ALTA
6	Llamadas	onTrack() llama rtc.setRemoteAudioTrack() pero rtc puede ser null si se destruyó	ALTA
7	Llamadas	setRemoteAnswer() se llama desde hilo Firestore — puede causar race condition en WebRTC	MEDIA
8	Mensajes	listenConversation() puede emitir listas desordenadas si 2 snapshots llegan juntos	MEDIA
9	Mensajes	sendMessage() no maneja el caso de otherUid vacío — mensaje enviado a sí mismo	MEDIA

1. Auditoría del Sistema de Login

El flujo de autenticación tiene 2 bugs adicionales no detectados anteriormente:

✗ BUG 1 — reloadUser() sin await real — puede no sincronizar

Problem a: En AuthViewModel.login(), la llamada es: repo.reloadUser() sin capturar su resultado. reloadUser() es una suspend function que puede tardar 200-500ms en red lenta. Si la red es lenta, isEmailVerified puede leerse ANTES de que reload() complete. El usuario ve 'Email no verificado' aunque ya lo verificó.

Solución : Esperar explícitamente: repo.reloadUser() sin ignorar el resultado. Adicionalmente, agregar un reintento si falla la sincronización.

■ BUG 2 — Usuario Google va a ChooseNumber aunque ya tiene número

Problem a: PermissionActivity llama userRepo.getUser(uid) para decidir si ir a ChooseNumber o MainActivity. Si el usuario inicia sesión con Google por primera vez Y ya tenía cuenta (email vinculado), el documento en Firestore existe pero puede tardar en cargarse. Si getUser() devuelve null por timeout de red, va a ChooseNumber y le pide número de nuevo.

Solución : Agregar retry con delay en getUser(). Si falla 3 veces, ir a MainActivity directamente (el número se puede elegir más tarde desde el perfil).

AuthViewModel.kt — login() corregido con await explícito

ui/auth/AuthViewModel.kt — fun login() CORREGIDO

```
fun login(email: String, pass: String) = viewModelScope.launch {
    _state.value = AuthUiState.Loading
    val r = repo.loginWithEmail(email, pass)
    if (r.isFailure) {
        _state.value = AuthUiState.Error(translate(r.exceptionOrNull()?.message))
        return@launch
    }
    val user = r.getOrNull()!!
    // FIX: await explícito del reload – esperar resultado real
    // Si falla la red, continuar igual (el token local puede ser suficiente)
    try { repo.reloadUser() } catch (_: Exception) { /* continuar con token local */ }
    val isPwd = user.providerData.any { it.providerId == "password" }
    // Re-leer isEmailVerified DESPUÉS del reload
    val verified = auth.currentUser?.isEmailVerified ?: false
    if (isPwd && !verified) {
        repo.logout()
        _state.value = AuthUiState.NeedsVerification(email)
    } else {
        _state.value = AuthUiState.LoggedIn(user.uid)
    }
}

// Agregar campo en AuthViewModel (necesita acceso al auth para re-leer):
private val auth = com.google.firebase.auth.FirebaseAuth.getInstance()
```

PermissionActivity.kt — proceed() con retry

ui/auth/PermissionActivity.kt — fun proceed() CORREGIDO

```
private fun proceed() {
    lifecycleScope.launch {
```

```
val uid = FirebaseAuth.getInstance().currentUser?.uid ?: return@launch
// FIX: retry hasta 3 veces con delay para usuarios Google con red lenta
var user = userRepo.getUser(uid)
if (user == null) {
    kotlinx.coroutines.delay(500)
    user = userRepo.getUser(uid)
}
if (user == null) {
    kotlinx.coroutines.delay(1000)
    user = userRepo.getUser(uid)
}
// Si sigue null después de 3 intentos: ir a MainActivity
// (puede ser usuario nuevo o error de red temporal)
val dest = when {
    user?.phoneNumber?.isNotBlank() == true -> MainActivity::class.java
    user == null -> MainActivity::class.java // error de red – no forzar número
    else -> ChooseNumberActivity::class.java
}
startActivity(Intent(this@PermissionActivity, dest))
finish()
}
}
```

2. Auditoría del Sistema de Llamadas

Este es el área más crítica. Se encontraron 5 bugs que afectan directamente que la llamada suene, se conecte, tenga buen audio y se pueda cortar.

✗ BUG 3 — CRÍTICO: FCM no incluye el SDP offer — receptor no puede contestar

Problem a: MyFirebaseMessagingService.handleCall() recibe el callId y callerUid pero el 'offer' SDP viene en data['offer']. El problema real: la app NUNCA ENVÍA la notificación FCM. La guía muestra cómo RECIBIR la notificación pero nunca explica cómo ENVIARLA. Sin un servidor (Firebase Cloud Functions o backend propio) que envíe el FCM push, el receptor NUNCA recibe la llamada si la app está cerrada. El ForegroundService y la señalización Firestore solo funcionan si la app está abierta.

Solución : Para que las llamadas lleguen con la app cerrada se necesita Firebase Cloud Functions. Mientras tanto: la app debe escuchar llamadas entrantes via Firestore listener en un Service permanente. Ver código completo en sección 2.1.

✗ BUG 4 — ForegroundService arranca antes del offer — ringtone sin llamada real

Problem a: En MyFirebaseMessagingService.handleCall(), se llama startForegroundService() SIEMPRE que llega una notificación FCM. Si la notificación llega pero el offer SDP en Firestore no está disponible aún (race condition de red), el ringtone suena pero createAnswer() falla porque remoteSdp está vacío.

Solución : Verificar que el offer SDP es válido antes de iniciar el ForegroundService. El offer ya llega en el FCM data payload — validarlo antes de proceder.

✗ BUG 5 — Sin AudioFocus — audio del sistema interfiere con la llamada

Problem a: WebRTC maneja su propio audio PERO Android requiere que la app solicite AudioFocus para interrumpir música, podcasts u otras apps de audio. Sin esto: si el usuario tiene música sonando, ambos audios se mezclan y se escucha horrible. También puede causar que el auricular y el altavoz no respondan correctamente.

Solución : Solicitar AudioFocus en CallActivity.setupAudio() y liberarlo en hangUp(). Ver código en sección 2.2.

✗ BUG 6 — onTrack() accede a rtc que puede ser null

Problem a: En CallActivity, el observer de PeerConnection se crea con una lambda que captura 'rtc'. Si onTrack() se llama desde un hilo de WebRTC DESPUÉS de que hangUp() llamó rtc.disconnect(), rtc.setRemoteAudioTrack() lanza NullPointerException. Esto puede ocurrir en cierres de llamada rápidos o al recibir tracks tardíos.

Solución : Verificar que rtc no es null antes de llamar setRemoteAudioTrack(). Usar 'rtc?.setRemoteAudioTrack(it)' con safe call.

■ BUG 7 — setRemoteAnswer() desde hilo Firestore — race condition WebRTC

Problem a: listenForAnswer() dispara el callback desde el hilo principal de Firestore listeners. setRemoteAnswer() → peerConnection.setRemoteDescription() debe llamarse desde un hilo específico en algunas versiones de WebRTC. En Android, PeerConnection es thread-safe para la mayoría de operaciones, pero setRemoteDescription después de createOffer puede crear una carrera si ICE candidates llegan antes de que setRemoteDescription complete.

Solución : Ejecutar setRemoteAnswer() en el hilo principal con runOnUiThread o en un Dispatcher.Main. Esto garantiza el orden de operaciones correcto.

2.1 — Solución para llamadas con app cerrada: IncomingCallService

Sin Cloud Functions, la única forma de recibir llamadas con la app cerrada es un Foreground Service permanente que escuche Firestore. Este Service se inicia cuando el usuario hace login y escucha llamadas entrantes.

¡ OPCIÓN RECOMENDADA A CORTO PLAZO: El receptor debe tener la app abierta para recibir llamadas. Esto es lo mismo que Telegram si no tiene notificaciones configuradas. Funciona perfectamente en pruebas con 2 teléfonos donde ambos tienen la app abierta.

service/IncomingCallListenerService.kt — ARCHIVO NUEVO

```
package com.simplecallapp.service

import android.app.*; import android.content.*; import android.os.*
import androidx.core.app.NotificationCompat
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.firestore.FirebaseFirestore
import com.google.firebase.firestore.ListenerRegistration
import com.simplecallapp.ui.call.CallActivity

// Este service escucha llamadas entrantes aunque la app esté en segundo plano.
// Iniciar en MainActivity.onCreate() si el usuario está logueado.
class IncomingCallListenerService : Service() {

    companion object {
        const val CHANNEL_ID = "incoming_listener"
        const val NOTIF_ID = 2001
    }

    private var firestoreListener: ListenerRegistration? = null
    private val processedCalls = mutableSetOf<String>()

    override fun onCreate() {
        super.onCreate()
        createChannel()
        startForeground(NOTIF_ID, buildNotif())
        startListening()
    }

    private fun startListening() {
        val uid = FirebaseAuth.getInstance().currentUser?.uid ?: return
        firestoreListener = FirebaseFirestore.getInstance()
            .collection("calls")
            .whereEqualTo("receiverUid", uid)
            .whereEqualTo("status", "ringing")
            .addSnapshotListener { snap, _ ->
                snap?.documentChanges?.forEach { change ->
                    val callId = change.document.getString("callId") ?: return@forEach
                    if (processedCalls.contains(callId)) return@forEach
                    processedCalls.add(callId)
                    val callerUid = change.document.getString("callerUid") ?: return@forEach
                    val callerNum = change.document.getString("callerNumber") ?: "Desconocido"
                    val offer = change.document.getString("offer") ?: return@forEach
                    if (offer.isNotBlank()) showIncomingCall(callId, callerUid, callerNum, offer)
                }
            }
    }

    private fun showIncomingCall(callId:String, callerUid:String, callerNum:String, offer:String) {
        val intent = Intent(this, CallActivity::class.java).apply {
            flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TOP
        }
    }
}
```

```

putExtra("CALL_ID", callId)
putExtra("IS_CALLER", false)
putExtra("REMOTE_UID", callerUid)
putExtra("REMOTE_NAME", callerNum)
putExtra("REMOTE_SDP", offer)
}

val pi = PendingIntent.getActivity(this, callId.hashCode(), intent,
PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE)
val notif = NotificationCompat.Builder(this, "calls")
    .setSmallIcon(android.R.drawable.ic_menu_call)
    .setContentTitle("Llamada entrante")
    .setContentText("De: $callerNum")
    .setPriority(NotificationCompat.PRIORITY_MAX)
    .setCategory(NotificationCompat.CATEGORY_CALL)
    .setFullScreenIntent(pi, true)
    .setAutoCancel(true).setOngoing(true).build()
(getSystemService(NOTIFICATION_SERVICE) as NotificationManager)
    .notify(callId.hashCode(), notif)
// Iniciar ringtone
startService(Intent(this, CallForegroundService::class.java).apply {
putExtra(CallForegroundService.EXTRA_REMOTE_NAME, "De: $callerNum") })
}

override fun onDestroy() { super.onDestroy(); firestoreListener?.remove() }
override fun onBind(i: Intent?): IBinder? = null

private fun buildNotif() = NotificationCompat.Builder(this, CHANNEL_ID)
    .setSmallIcon(android.R.drawable.ic_menu_call)
    .setContentTitle("SimpleCallApp activo")
    .setContentText("Esperando llamadas entrantes")
    .setPriority(NotificationCompat.PRIORITY_MIN)
    .setOngoing(true).build()

private fun createChannel() {
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
val nm = getSystemService(NOTIFICATION_SERVICE) as NotificationManager
nm.createNotificationChannel(NotificationChannel(
CHANNEL_ID, "SimpleCallApp", NotificationManager.IMPORTANCE_LOW))
}
}
}

```

¡ Agregar en AndroidManifest.xml dentro de :

¡ Agregar en MainActivity.onCreate(): if (FirebaseAuth.getInstance().currentUser != null) {
startService(Intent(this, IncomingCallListenerService::class.java)) }

2.2 — AudioFocus corregido en CallActivity

ui/call/CallActivity.kt — setupAudio() y hangUp() CORREGIDOS

```

// Agregar campo en CallActivity:
private var audioFocusRequest: android.media.AudioFocusRequest? = null

// Reemplazar setupAudio():
private fun setupAudio() {
val am = getSystemService(AUDIO_SERVICE) as android.media.AudioManager

```

```

am.mode = android.media.AudioManager.MODE_IN_COMMUNICATION
am.isSpeakerphoneOn = false
// FIX: Solicitar AudioFocus para interrumpir música y otras apps
if (android.os.Build.VERSION.SDK_INT >= android.os.Build.VERSION_CODES.O) {
    audioFocusRequest = android.media.AudioFocusRequest.Builder(
        android.media.AudioManager.AUDIOFOCUS_GAIN_TRANSIENT
    )
        .setAudioAttributes(android.media.AudioAttributes.Builder()
            .setUsage(android.media.AudioAttributes.USAGE_VOICE_COMMUNICATION)
            .setContentType(android.media.AudioAttributes.CONTENT_TYPE_SPEECH)
            .build())
        .setOnAudioFocusChangeListener { }
        .build()
    am.requestAudioFocus(audioFocusRequest!!)
} else {
    @Suppress("DEPRECATION")
    am.requestAudioFocus(null,
        android.media.AudioManager.STREAM_VOICE_CALL,
        android.media.AudioManager.AUDIOFOCUS_GAIN_TRANSIENT)
}

// En hangUp(), ANTES de finish(), agregar:
// Liberar AudioFocus al terminar la llamada
val am = getSystemService(AUDIO_SERVICE) as android.media.AudioManager
am.mode = android.media.AudioManager.MODE_NORMAL
if (android.os.Build.VERSION.SDK_INT >= android.os.Build.VERSION_CODES.O) {
    audioFocusRequest?.let { am.abandonAudioFocusRequest(it) }
} else {
    @Suppress("DEPRECATION")
    am.abandonAudioFocus(null)
}

```

2.3 — onTrack() con null-safety y setRemoteAnswer() en hilo principal

ui/call/CallActivity.kt — setupWebRTC() CORREGIDO

```

// En setupWebRTC(), corregir onTrack():
override fun onTrack(t: RtpTransceiver?) {
    t?.receiver?.track()?.let { track ->
        if (track is AudioTrack) {
            // FIX: null-safety – rtc puede ser null si hangUp() ya se llamó
            rtc?.setRemoteAudioTrack(track)
        }
    }
}

// En callerFlow(), corregir setRemoteAnswer para ejecutar en hilo principal:
sig.listenForAnswer(callId) { answerSdp ->
    // FIX: ejecutar en hilo principal para garantizar orden WebRTC
    runOnUiThread {
        rtc?.setRemoteAnswer(answerSdp)
    }
}

// Igual en receiverFlow() para createAnswer:
rtc.createAnswer(intent.getStringExtra("REMOTE_SDP")?:"") { sdp ->

```

```

lifecycleScope.launch {
    sig.sendAnswer(callId, sdp.description)
    sig.listenForIceCandidates(callId, true) { m, i, s ->
        // FIX: addIceCandidate en hilo principal
        runOnUiThread { rtc?.addIceCandidate(IceCandidate(m, i, s)) }
    }
    sig.listenForHangup(callId) {
        runOnUiThread { if (callState != CallState.ENDED) hangUp() }
    }
}

```

2.4 — sendOffer debe incluir callerNumber en Firestore

Para que el receptor sepa quién llama, el offer debe incluir el número del llamador. Modificar `WebRTCSignaling.sendOffer()` y la llamada en `callerFlow()`:

`webrtc/WebRTCSignaling.kt` — `sendOffer()` con `callerNumber`

```

// Agregar parámetro callerNumber:
suspend fun sendOffer(
    callId: String, callerUid: String, receiverUid: String,
    sdp: String, callerNumber: String // NUEVO parámetro
) {
    calls.document(callId).set(mapOf(
        "callId" to callId,
        "callerUid" to callerUid,
        "callerNumber" to callerNumber, // NUEVO: para mostrar en notificación
        "receiverUid" to receiverUid,
        "offer" to sdp,
        "status" to "ringing",
        "timestamp" to System.currentTimeMillis()
    )).await()
}

// En CallActivity.callerFlow(), actualizar la llamada a sendOffer:
// Antes de callerFlow(), cargar el número del llamador:
private var myNumber = ""

// En onCreate(), agregar:
lifecycleScope.launch {
    myNumber = com.simplecallapp.data.repository.UserRepository()
        .getUser(myUid)?.phoneNumber ?: ""
}

// En callerFlow():
sig.sendOffer(callId, myUid, remoteUid, sdp.description, myNumber)

```


3. Auditoría del Sistema de Mensajes

■ BUG 8 — listenConversation() puede emitir listas desordenadas

Problem a: ChatRepository usa dos listeners en paralelo (s1 para mensajes enviados, s2 para recibidos). Cuando ambos snapshots llegan casi simultáneamente (dentro de ~50ms), se hacen 2 llamadas a trySend() con listas parcialmente actualizadas. El resultado: el RecyclerView puede mostrar brevemente mensajes en orden incorrecto o con un parpadeo visual.

Solución : Usar un Mutex para sincronizar las actualizaciones al mapa compartido, o combinar los dos listeners en un solo query con in() cuando Firestore lo permita. Solución pragmática: agregar un debounce de 50ms antes de emitir.

■ BUG 9 — sendMessage() sin validar otherUid vacío

Problem a: Si ChatFragment se abre con otherUid="" (por un bug de navegación o args faltantes), sendMessage() envía un mensaje con to=". Este mensaje nunca aparece en ninguna conversación y ocupa espacio en Firestore.

Solución : Validar otherUid al inicio de sendMessage() y deshabilitar el botón Enviar si está vacío.

ChatRepository.kt — listenConversation() con sincronización

data/repository/ChatRepository.kt — listenConversation() CORREGIDO

```
import kotlinx.coroutines.sync.Mutex
import kotlinx.coroutines.sync.withLock

fun listenConversation(myUid: String, otherUid: String): Flow<List<Message>> =
    callbackFlow {
        val merged = mutableMapOf<String, Message>()
        val mutex = Mutex()

        suspend fun emit() {
            val sorted = mutex.withLock { merged.values.sortedBy { it.timestamp } }
            trySend(sorted)
        }

        val s1 = col.whereEqualTo("from", myUid).whereEqualTo("to", otherUid)
            .orderBy("timestamp", Query.Direction.ASCENDING)
            .addSnapshotListener { snap, _ ->
                kotlinx.coroutines.runBlocking {
                    mutex.withLock {
                        snap?.toObjects(Message::class.java)?.forEach { merged[it.id] = it }
                    }
                }
                emit()
            }

        val s2 = col.whereEqualTo("from", otherUid).whereEqualTo("to", myUid)
            .orderBy("timestamp", Query.Direction.ASCENDING)
            .addSnapshotListener { snap, _ ->
                kotlinx.coroutines.runBlocking {
                    mutex.withLock {
                        snap?.toObjects(Message::class.java)?.forEach { merged[it.id] = it }
                    }
                }
                emit()
            }
    }
```

```
awaitClose { s1.remove(); s2.remove() }  
}
```

ChatFragment.kt — sendMessage() con validación

ui/chat/ChatFragment.kt — sendMessage() CORREGIDO

```
private fun sendMessage() {  
    val text = b.etMessage.text?.toString()?.trim() ?: ""  
    if (text.isBlank()) return  
    // FIX BUG 9: validar que otherUid no está vacío  
    if (otherUid.isBlank()) {  
        Toast.makeText(requireContext(),  
            "Error: destinatario inválido", Toast.LENGTH_SHORT).show()  
        return  
    }  
    if (myNumber.isBlank()) {  
        Toast.makeText(requireContext(),  
            "Cargando tu número...", Toast.LENGTH_SHORT).show()  
        return  
    }  
    lifecycleScope.launch {  
        try {  
            chatRepo.sendMessage(Message(  
                from = myUid,  
                fromNumber = myNumber,  
                to = otherUid,  
                toNumber = otherNumber,  
                text = text,  
                delivered = true  
            ))  
            b.etMessage.setText("")  
        } catch (e: Exception) {  
            Toast.makeText(requireContext(),  
                "Error al enviar: ${e.message}", Toast.LENGTH_SHORT).show()  
        }  
    }  
}
```

4. Flujos Correctos Después de las Correcciones

Flujo de Login (email) — Correcto

Usuario	Escribe email + contraseña → toca 'Iniciar sesión'
AuthViewModel	signInWithEmailAndPassword → OK, obtiene FirebaseUser
AuthViewModel	repo.reloadUser() → espera respuesta del servidor
AuthViewModel	Re-lee auth.currentUser?.isEmailVerified (token actualizado)
AuthViewModel	Si verificado → emite LoggedIn(uid)
LoginActivity	Recibe LoggedIn → startActivity(PermissionActivity)
PermissionActivity	Verifica permisos → si OK, llama proceed()
PermissionActivity	userRepo.getUser(uid) con retry → navega a MainActivity
MainActivity	Inicia IncomingCallListenerService para escuchar llamadas

Flujo de Llamada Completo — Correcto

Llamador	Marca número → DialFragment.doCall() → crea callId y abre CallActivity
CallActivity	setupAudio(): MODE_IN_COMMUNICATION + AudioFocus
CallActivity	setupWebRTC(): crea WebRTCClient, initializePeerConnection(), addLocalAudioTrack()
CallActivity	callerFlow(): rtc.createOffer() → callback con SDP
CallActivity	sig.sendOffer(callId, myUid, remoteUid, sdp, myNumber) → guarda en Firestore
Firestore	Documento 'calls/callId' creado con status='ringing' + offer + callerNumber
IncomingCallListenerService	Detecta nuevo documento con receiverUid=miUid y status=ringing
IncomingCallListenerService	Muestra notificación pantalla completa + inicia ringtone
Receptor	Toca notificación → abre CallActivity con IS_CALLER=false y REMOTE_SDP=offer
CallActivity (receptor)	setupAudio() + setupWebRTC() + receiverFlow()
CallActivity (receptor)	rtc.createAnswer(remoteSdp) → setRemoteDescription → createAnswer
CallActivity (receptor)	sig.sendAnswer(callId, answerSdp) → actualiza Firestore
Firestore	Documento actualizado: answer + status='connected'
Llamador	listenForAnswer detecta answer → runOnUiThread { rtc.setRemoteAnswer() }
Ambos	ICE candidates intercambiados via Firestore (solo ADDED)
Ambos	onConnectionChange(CONNECTED) → estado verde + timer + para ringtone
onTrack()	AudioTrack remoto capturado → rtc?.setRemoteAudioTrack(track)
Audio	Conversación bidireccional con codec Opus
Colgar	hangUp(): sig.endCall() + saveHistory() en cleanupScope + liberar AudioFocus

✓ El flow completo incluye: AudioFocus + IncomingCallListenerService + sendOffer con callerNumber + ICE solo ADDED + setRemoteAnswer en hilo principal + onTrack con null-safety + cleanupScope para historial. Con todo esto la llamada debería conectar sin retraso y con audio claro.

5. Orden de Implementación

1. Crear service/IncomingCallListenerService.kt (sección 2.1)
2. Agregar el service en AndroidManifest.xml con foregroundServiceType='phoneCall'
3. MainActivity.onCreate(): startService(IncomingCallListenerService) si está logueado
4. AuthViewModel.kt: corregir login() con re-lectura de isEmailVerified (sección 1)
5. PermissionActivity.kt: agregar retry en proceed() (sección 1)
6. WebRTCSignaling.kt: agregar callerNumber en sendOffer()
7. CallActivity.kt: setupAudio() con AudioFocus + null-safety en onTrack() + runOnUiThread en setRemoteAnswer (sección 2.3)
8. CallActivity.kt: hangUp() — agregar liberación de AudioFocus
9. ChatRepository.kt: listenConversation() con Mutex (sección 3)
10. ChatFragment.kt: sendMessage() con validación de otherUid (sección 3)
11. Build → Clean → Rebuild → probar con 2 dispositivos físicos

✓ Las correcciones más importantes para que la llamada funcione bien son: IncomingCallListenerService (para recibir llamadas), AudioFocus (para calidad de audio), y callerNumber en sendOffer (para mostrar quién llama en la notificación).